ENSA BM

Architecture Logicielle

Rapport de Travaux Pratiques

TP1

Du Monolithe Simple à l'Architecture Distribuée

Encadrant : Pr. ELBAGHAZAOUI

Année universitaire : 2025 / 2026

TP1 — Application Monolithique Simple

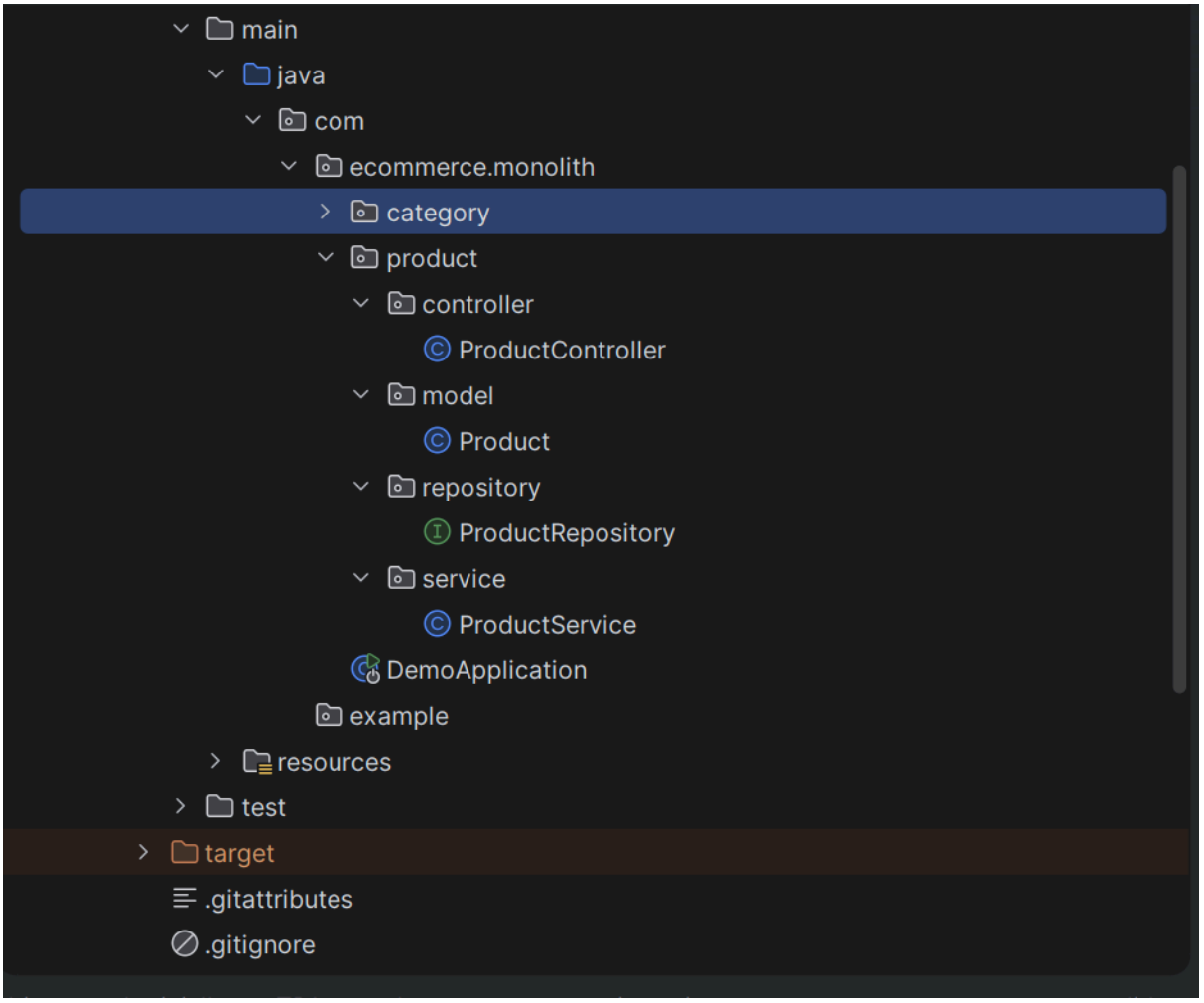
1.1 Objectifs

- Créer une application Spring Boot monolithique simple
- Développer une API REST CRUD complète
- Comprendre l'architecture en couches (Entity, Repository, Service, Controller)
- Découvrir RestRepository pour simplifier le code
- Identifier les avantages et limites d'un monolithe

1.2 Outils utilisés

Outil	Version	Rôle
Java	17+	Langage principal
Spring Boot	3.x	Framework applicatif
Spring Data JPA	-	Accès base de données
PostgreSQL	-	Base de données relationnelle
Lombok	-	Réduction du boilerplate
Maven	3.8+	Gestionnaire de dépendances
Postman	-	Tests des endpoints

1.3 Structure du projet



1.4 Entité Product

L'entité principale du TP1 est Product, avec les champs suivants :

Champ	Type	Contrainte
id	Long	@GeneratedValue IDENTITY
name	String	@NotBlank, @Column(nullable=false)
description	String	@Column(length=1000)
price	BigDecimal	@NotNull, @Positive
stock	Integer	@NotNull, @PositiveOrZero

1.5 Exercice — Entité Category

L'exercice du TP1 demandait d'ajouter une entité Category avec une relation ManyToOne avec Product.

```java
Category.java   Product.java   ProductService.java   CategoryRepository.java   ProductController.jav

public class Product {

    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;

    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
}
```

```java
Category.java

package com.ecommerce.monolith.category.model;
import ...

@Entity   laarichothman-prog
@Table(name = "categories")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Category {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Category name is required")
    private String name;
}
```

1.6 Tests Postman

Méthode	URL	Description	Status
GET	/api/products	Liste tous les produits	200
POST	/api/products	Créer un produit	201
PUT	/api/products/{id}	Modifier un produit	200
DELETE	/api/products/{id}	Supprimer un produit	204

http://localhost:8080/api/products

Save Share

POST http://localhost:8080/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Laptop Dell XPS",
3   "description": "High performance laptop",
4   "price": 999.99,
5   "stock": 10
6 }
```

Body 201 Created 681 ms 284 B

{} JSON Preview Visualize

```
1 {
2     "id": 2,
3     "name": "Laptop Dell XPS",
4     "description": "High performance laptop",
5     "price": 999.99,
6     "stock": 10,
7     "category": null
8 }
```

Search

Overview GET http://localhost:8080/a No enviro...

http://localhost:8080/api/products

Save Share

GET http://localhost:8080/api/products Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Document this request... Write with AI

Body 200 OK 366 ms 391 B

{} JSON Preview Visualize

```
1 [
2     {
3         "id": 1,
4         "name": "Laptop Pro",
5         "description": "Upgraded gaming laptop",
6         "price": 1500.00,
7         "stock": 5,
8         "category": null
```